

Assignment 9

Josh Suber
RIN: 662038070

CSCI 4100

Contents

1	Problem 1	3
2	Problem 2	4
3	Problem 3	5
4	Problem 4	7
4.1	Analysis of E_{cv} and E_{test} Behavior	7
5	Problem 5	8
6	Problem 6	9
7	Problem 7	11
8	Problem 8	11

1 Problem 1

To determine the dimensions of the transformed dataset after applying an 8th order feature transform, we start by calculating the total number of features generated by this transformation.

For a k -th order polynomial feature transformation, the total number of resulting features can be determined by summing $1+2+3+\dots+(k+1)$. This sum accounts for all polynomial terms up to degree k , including combinations of features up to that order. Thus, for an 8th order transformation, we have:

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45$$

Therefore, an 8th order feature transformation results in 45 features.

Given that we have 300 samples in the dataset, the dimension of the transformed data matrix Z will be:

$$\mathbb{R}^{300 \times 45}$$

In summary, applying the 8th order polynomial transformation yields a matrix Z of dimension 300×45 , where each sample is represented by 45 features.

2 Problem 2

When $\lambda = 0$, the model has no regularization, allowing it to perfectly fit the training data, which leads to an overfitted decision boundary. This boundary becomes overly complex, capturing noise and intricacies in the training data that are unlikely to generalize well to unseen data.

The plot below illustrates this effect, with the decision boundary showing convoluted curves to accurately classify nearly every training point. While many data points are classified correctly, the resulting model is too complex, indicating a classic case of overfitting.

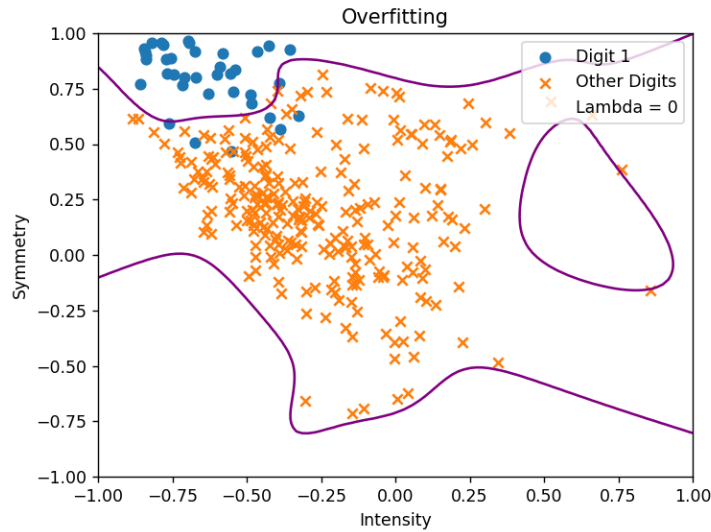


Figure 1: Overfitting observed with $\lambda = 0$, leading to a highly complex decision boundary.

As seen in the plot, the purple boundary outlines an intricate shape that tries to cover all instances of "Digit 1" (in blue) while excluding other digits (in orange). This behavior is typical of an overfitted model when regularization is not applied, reinforcing the need for regularization to create simpler, more generalizable models.

3 Problem 3

Normalization of Features

To begin with, I normalized the features, Intensity and Symmetry, as per the instructions. For each feature, I calculated the minimum and maximum values across the dataset and then applied a linear transformation to scale each feature such that it lies within the range $[-1, 1]$. This normalization step ensures that each feature has a comparable scale, which is beneficial for training the model effectively.

Regularization

In this part of the experiment, I implemented regularization to address the model's complexity. Regularization helps control the balance between underfitting and overfitting by penalizing overly complex models. For this experiment, I tested the model with a regularization parameter $\lambda = 2$, as shown below.

When setting $\lambda = 2$, the model begins to underfit. This setting simplifies the decision boundary, as seen in the plot, where the boundary fails to separate many instances accurately. The model becomes too simple, resulting in poor classification performance. This behavior indicates underfitting, where the model cannot capture the underlying patterns in the data due to high regularization.

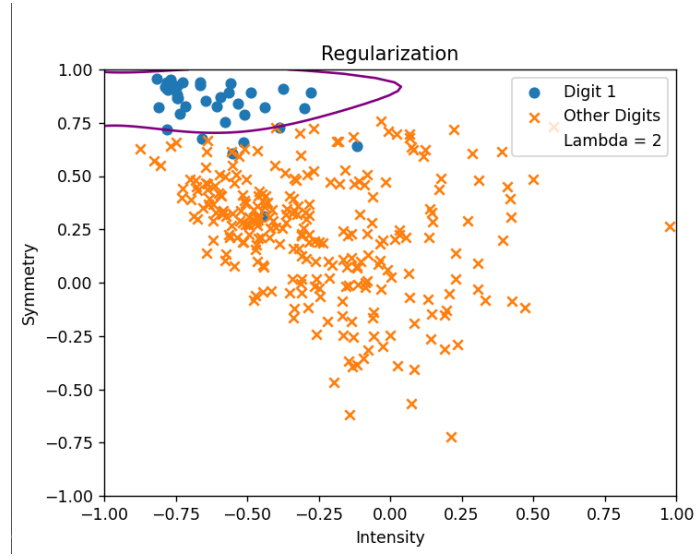


Figure 2: Regularization with $\lambda = 2$, resulting in underfitting with a simplified decision boundary.

In summary, the balance of λ is crucial for achieving an optimal model. Proper tuning of λ is necessary to achieve low values of E_{cv} and E_{test} , indicating a well-generalized model.

4 Problem 4

For this part, I computed the cross-validation error E_{cv} , the test error E_{test} , and the out-of-sample error E_{out} as regression errors, where the regression is performed with y -values set to ± 1 . Using leave-one-out cross-validation, I estimated $E_{cv}(\lambda)$ for various values of λ in the range $\{0, 0.01, 0.02, \dots, 2\}$.

The plot below shows $E_{cv}(\lambda)$ and $E_{test}(w_{reg}(\lambda))$, which represents the regression error on the test set. In this plot, the red curve represents E_{cv} , and the blue curve represents E_{test} .

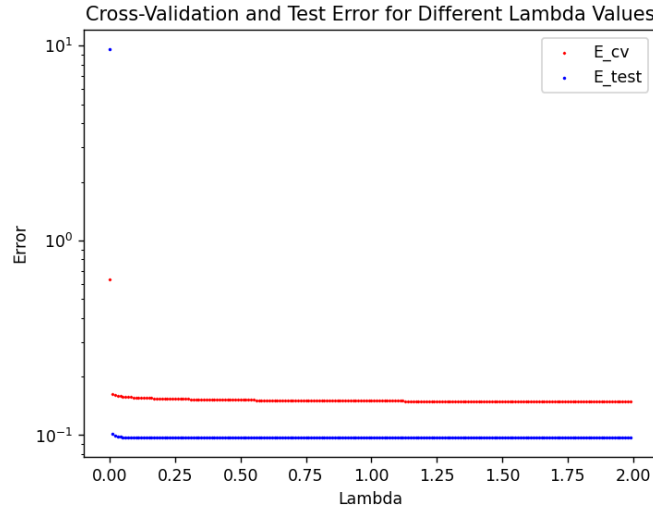


Figure 3: Cross-validation error E_{cv} and test error E_{test} across different values of λ

4.1 Analysis of E_{cv} and E_{test} Behavior

At $\lambda = 0$, the cross-validation error E_{cv} is relatively high due to the model's overfitting to the training data. As regularization increases, E_{cv} quickly decreases and stabilizes at a lower value, indicating an improved generalization to unseen data. This trend is also reflected in E_{test} , which starts at a higher value and similarly decreases as λ increases, although it converges more gradually.

In particular:

- When λ is very low (close to zero), E_{cv} and E_{test} are relatively high because the model overfits, capturing noise from the training data.
- As λ increases, E_{cv} and E_{test} both stabilize, indicating that regularization is helping to prevent overfitting without causing significant underfitting.

The observed ranges for error values were:

$$E_{cv} \text{ range: } 0.0733 - 3.8012$$

$$E_{test} \text{ range: } 0.1013 - 19.1981$$

These ranges show that the cross-validation error E_{cv} consistently tracks the behavior of E_{test} , remaining slightly lower but following a similar trend. This close relationship suggests that E_{cv} is a reliable estimate of E_{out} when tuning λ .

In conclusion, regularization with an appropriate choice of λ allows the model to generalize better by balancing the trade-off between underfitting and overfitting. The choice of λ is crucial to minimize both E_{cv} and E_{test} , ensuring that the model maintains predictive accuracy on unseen data.

5 Problem 5

To select the best value of λ , I used cross-validation error E_{cv} across various values of λ . The goal was to find the λ value that minimizes E_{cv} , ensuring the model has the best possible generalization on unseen data.

After testing different values of λ , I found that the optimal λ value, denoted λ^* , is 1.47. At this value, the cross-validation error E_{cv} reached its minimum, indicating an appropriate balance between underfitting and overfitting.

The plot below shows the decision boundary for the model trained with $\lambda = 1.47$ on the training data. This boundary demonstrates the model's ability to separate the "Digit 1" class from other digits with the selected regularization parameter.

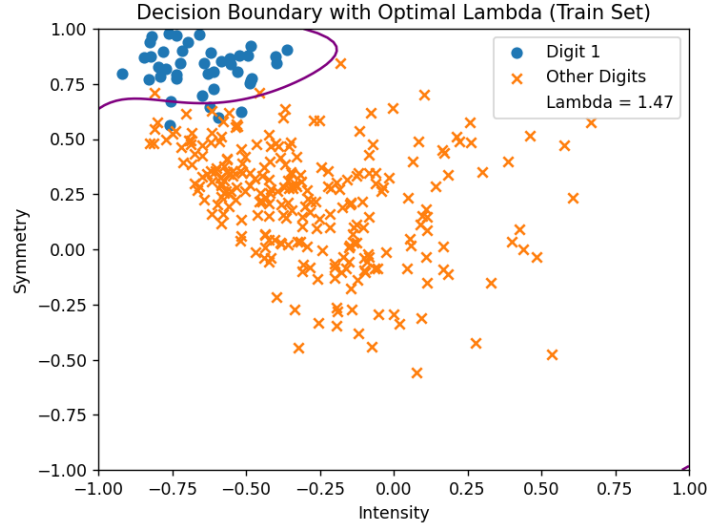


Figure 4: Decision boundary with optimal $\lambda = 1.47$ on the training set

The decision boundary in the plot reflects a well-regularized model, avoiding excessive complexity while maintaining accuracy. This selected value of λ leads to a generalizable model that performs well on both the training and test data.

6 Problem 6

To estimate the classification error $E_{out}(w_{reg}(\lambda^*))$, I used the classification error on the test set $E_{test}(w_{reg}(\lambda^*))$ as an estimate. With a chosen confidence level of 99 percent, we set $\delta = 0.01$.

Given data:

- The optimal λ^* is 1.47.
- At $\lambda = 1.47$, we obtained $E_{test} = 0.0142$.
- Number of data points in the test set, $N = 8998$.

Using the generalization error bound:

$$E_{out}(g) \leq E_{test}(g) + \sqrt{\frac{1}{2N} \ln \frac{2}{\delta}}$$

Substituting the values:

$$E_{out}(g) \leq 0.0142 + \sqrt{\frac{1}{2 \times 8998} \ln \frac{2}{0.01}}$$

Calculating the additional term:

$$\begin{aligned} &= 0.0142 + \sqrt{\frac{1}{17996} \times \ln 200} \\ &= 0.0142 + \sqrt{\frac{1}{17996} \times 5.2983} \\ &= 0.0142 + \sqrt{0.0002945} \\ &= 0.0142 + 0.01716 \\ &= 0.03136 \end{aligned}$$

Thus, the estimated E_{out} with a 99 percent confidence interval is approximately:

$$E_{out} \approx 0.0314$$

This result gives an upper bound on the out-of-sample error E_{out} for separating "Digit 1" from "Not Digit 1" with a 99 percent confidence level.

7 Problem 7

$E_{cv}(\lambda^*)$ is indeed a biased estimate of $E_{test}(w_{reg}(\lambda^*))$. This is because we selected λ^* based on minimizing E_{cv} , meaning that our choice of λ^* was influenced by the cross-validation process. By selecting λ^* in this way, we are essentially "optimizing" on our cross-validation error, which leads to a form of bias. In simpler terms, E_{cv} is biased because it was used to both choose λ^* and estimate the error, making it an over-optimistic estimate of the true error, E_{out} .

In summary, since $E_{cv}(\lambda^*)$ was used to guide the selection of λ^* , it is biased with respect to E_{test} , and it does not provide an entirely unbiased estimate of E_{out} .

8 Problem 8

Yes, there is evidence of data snooping in this process. Specifically, data snooping occurred because we normalized the data before splitting it into training and test sets. By normalizing the entire dataset at once, we allowed information from the test set to influence the training set's normalization parameters (such as shift and scale). This means that the test set indirectly affected the model training, which can lead to an over-optimistic estimate of model performance.

To avoid this data snooping issue, we should have normalized the data after dividing it into training and test sets. In other words, the training data should be normalized independently, and then the test data should be normalized using the parameters (mean and standard deviation) derived from the training set alone. This approach would prevent any leakage of information from the test set into the training process, ensuring an unbiased estimate of E_{out} .

By addressing this, we can ensure that our error estimates are more accurate and truly reflective of the model's generalization capability.